# Machine Learning

Unidad 3 — Fundamentos matemáticos, algoritmos clásicos, evaluación y buenas prácticas.

# Contenido


01 Álgebra lineal para ML

02 Descenso de gradiente

03 Teorema de Bayes

04 Análisis exploratorio de datos (EDA)

05 Preprocesamiento de datos

06 Introducción a Machine Learning

07 Aprendizaje supervisado: regresión avanzada

08 Aprendizaje supervisado: clasificación avanzada

09 Aprendizaje no supervisado

10 Modelos de ensamble

11 Pipelines y evaluación de modelos

01

# Álgebra Lineal para ML

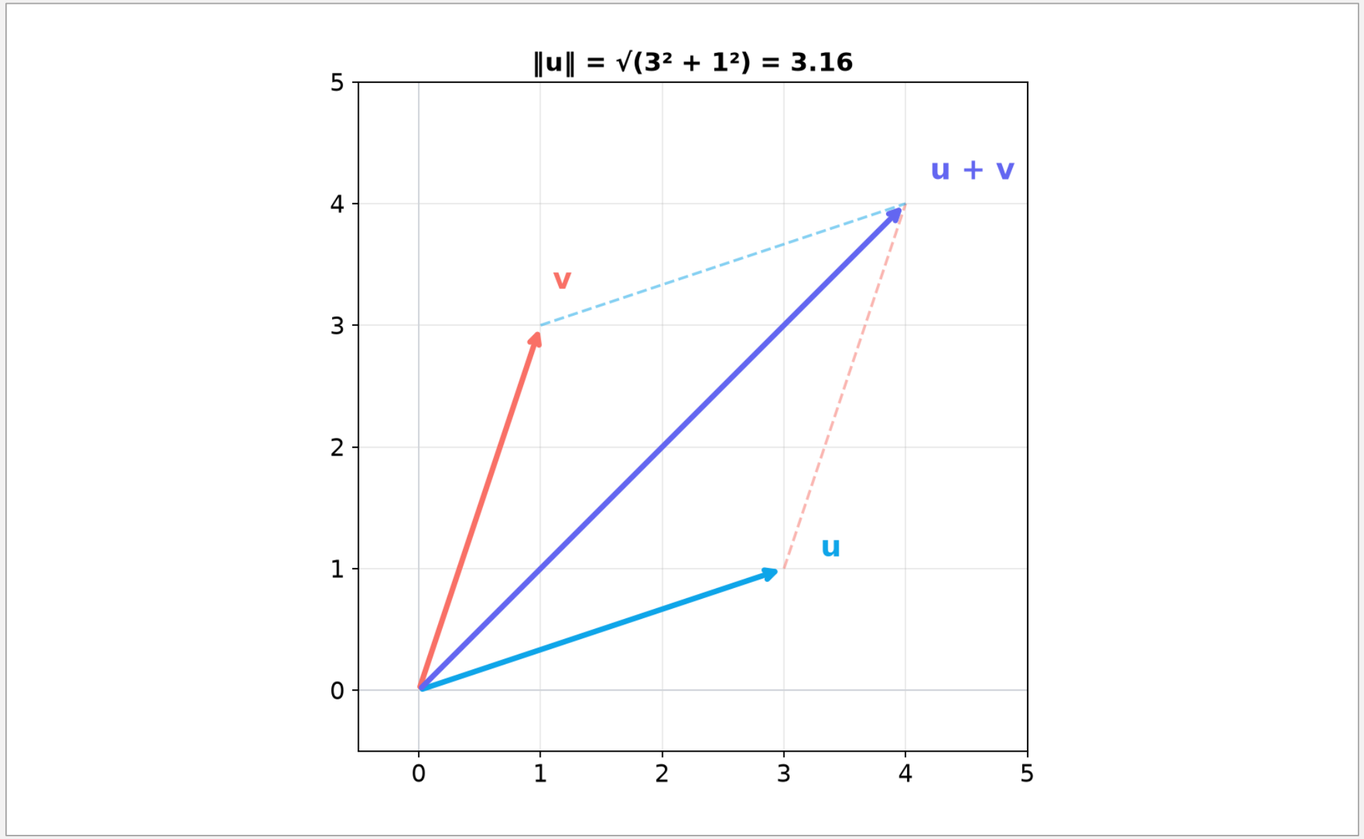
01

# Vectores y su norma

Un vector representa, en ML, una fila de datos: cada característica es una componente

Norma (magnitud): $\|v\| = \sqrt{v_1^2 + v_2^2 + \cdots + v_n^2}$

- Espacio vectorial: el conjunto de vectores que se pueden formar combinando una base
- Base: el conjunto mínimo de vectores que genera todo el espacio
- Dimensión: cuántos vectores tiene la base (en ML, el número de características)



# Operaciones con matrices

| Operación | Regla | Uso típico en ML |
|---|---|---|
| **Suma / resta** | Elemento a elemento, mismas dimensiones | Combinar o comparar conjuntos de pesos |
| **Producto punto** | $u \cdot v = \sum_i u_i v_i$ | Similitud entre dos vectores de características |
| **Producto matricial** | $(AB)_{ij} = \sum_k A_{ik} B_{kj}$ | Propagar datos a través de una capa de una red |
| **Producto matriz-vector** | $Ax$ = combinación lineal de las columnas de $A$ | Aplicar un modelo lineal a una fila de datos |
| **Transpuesta / inversa** | $A^{\mathrm{T}}$ intercambia filas y columnas; $A^{-1}$ deshace $A$ | Resolver sistemas de ecuaciones normales |

Toda la matemática de una red neuronal (Unidad 4) se reduce, en el fondo, a productos matriciales encadenados.

02

# Descenso de Gradiente

# Cómo funciona el descenso de gradiente

- Función de costo $J(\theta)$: mide qué tan lejos están las predicciones de los valores reales
- Entrenar = encontrar los parámetros $\theta$ que minimizan $J(\theta)$
- $\alpha$ (tasa de aprendizaje): qué tan grande es cada paso hacia el mínimo
- Ejemplo real del notebook: $J(\theta) = \theta^2 + 3\theta + 2$, mínimo en $\theta = -1.5$

---

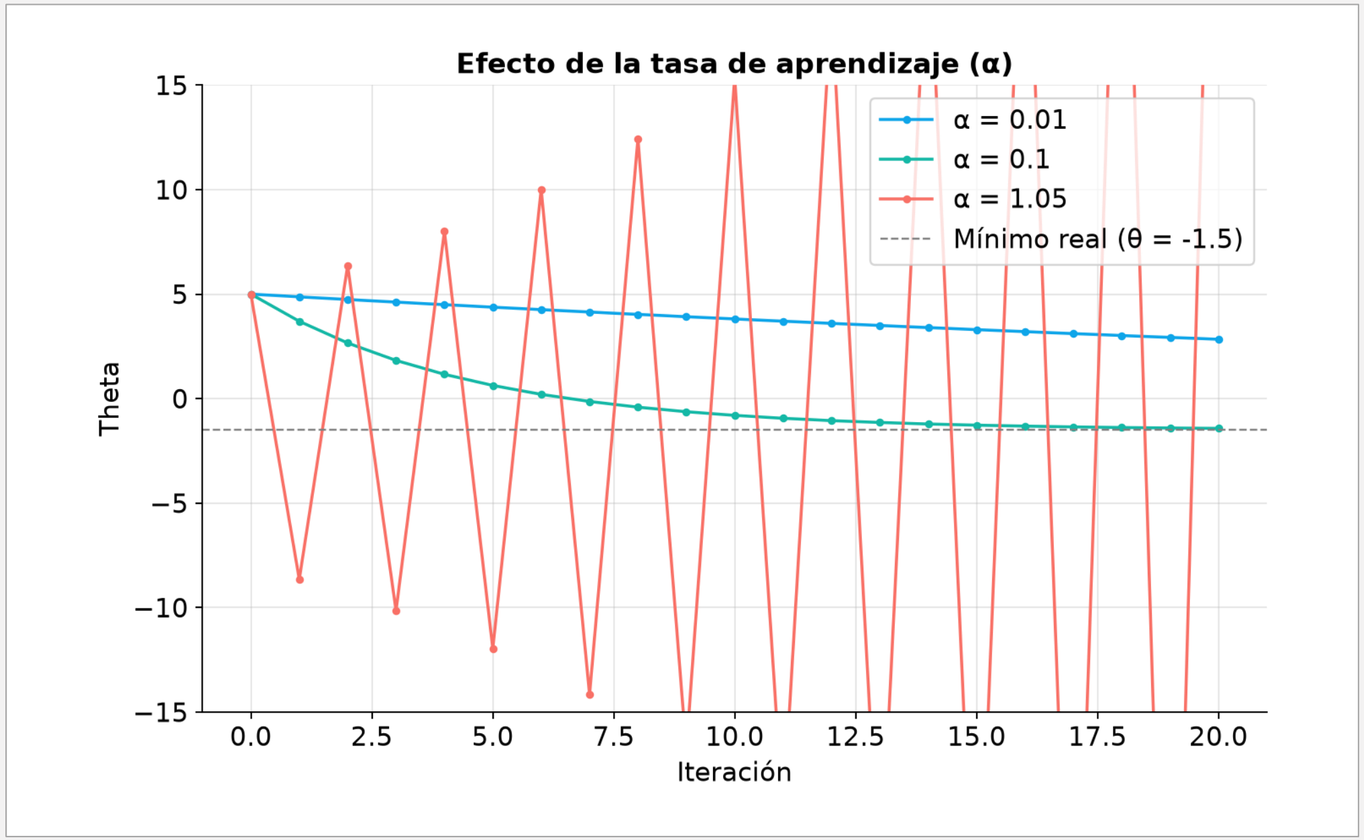
$$\theta_{nuevo} = \theta_{actual} - \alpha \cdot J'(\theta_{actual})$$

Regla de actualización del descenso de gradiente.

---

# El efecto real de la tasa de aprendizaje

- $\alpha = 0.01$: converge, pero muy lento — $\theta = 2.84$ tras 20 iteraciones, lejos aún de −1.5
- $\alpha = 0.1$: converge rápido y estable — $\theta = -1.43$, muy cerca del mínimo real
- $\alpha = 1.05$: diverge — oscila y crece sin control ($\theta = 42.2$, costo > 1900)

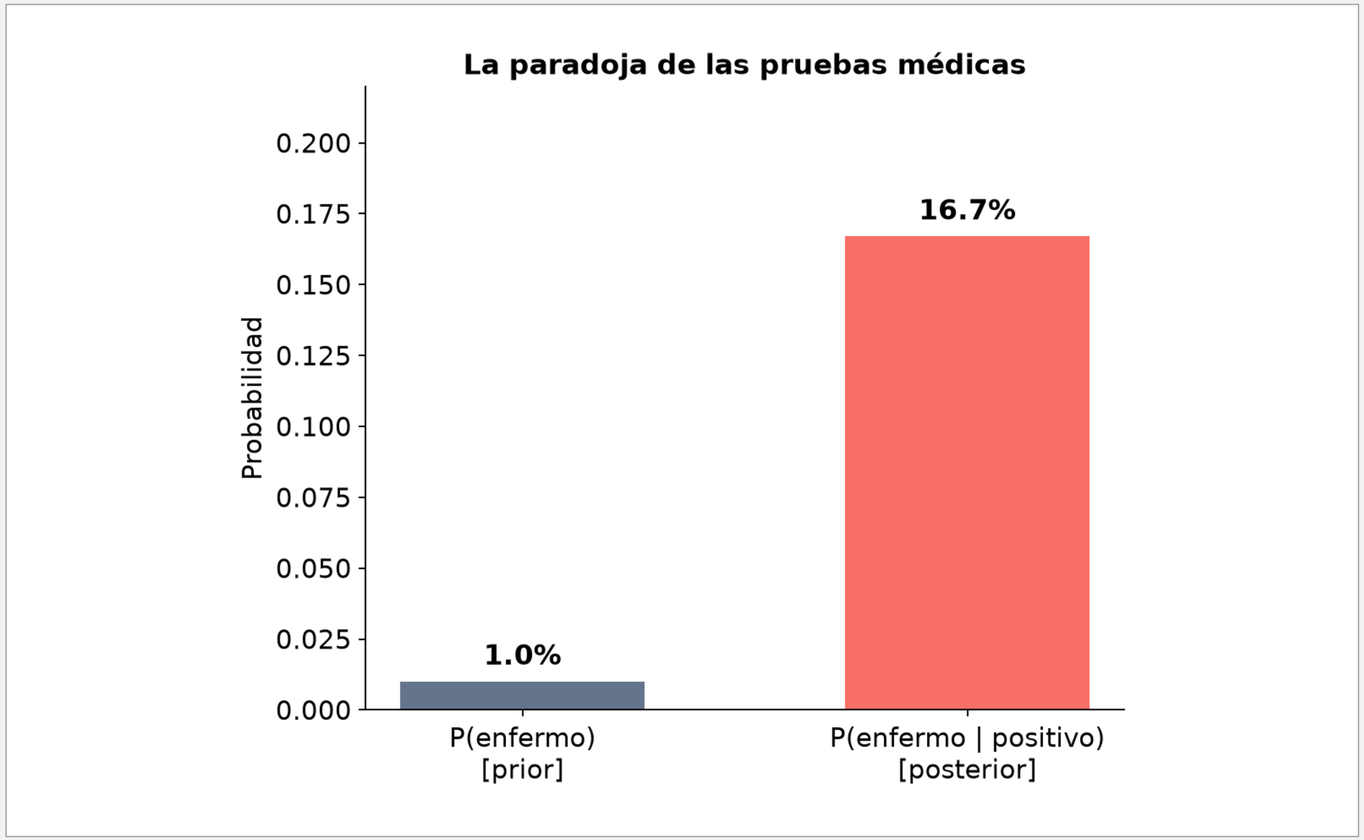


# 03

# Teorema de Bayes

# La paradoja de las pruebas médicas

- Prevalencia: 1% de la población tiene la enfermedad
- Sensibilidad: 99% de positivos entre los enfermos · Especificidad: 95% de negativos entre los sanos
- La mayoría intuye "90-95%" — el resultado real es 16.7%
- Como la enfermedad es rara, la mayoría de los positivos son falsos positivos del enorme grupo de personas sanas



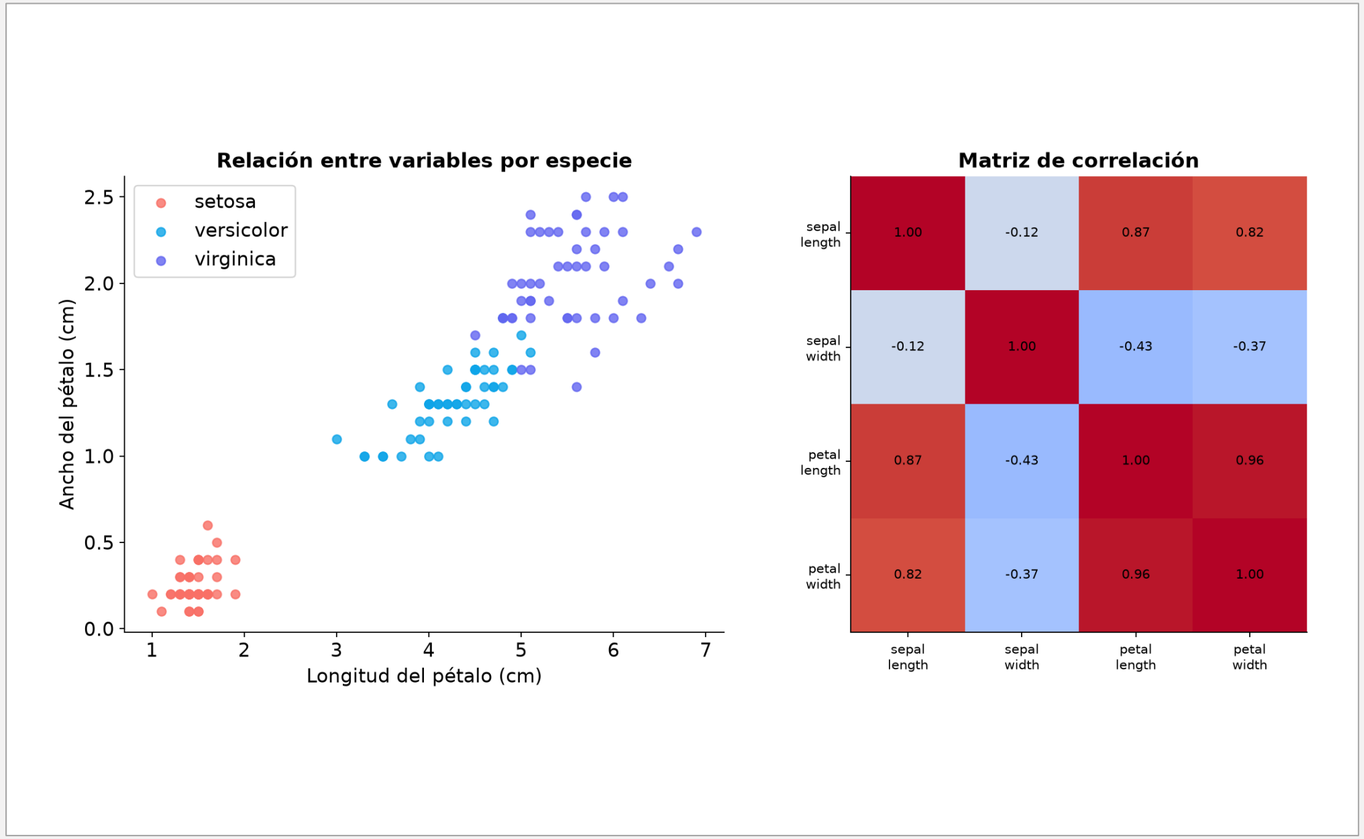
04

# Análisis Exploratorio de Datos (EDA)

04

# El flujo de trabajo de un EDA, sobre Iris

- Flujo: cargar → verificar (nulos, tipos) → describir → visualizar → correlacionar
- Las 3 especies se separan casi perfectamente por longitud y ancho del pétalo
- Se confirma en la matriz de correlación (0.96 entre ambas variables)



05

# Preprocesamiento de Datos

# Limpieza, outliers y transformación

| Problema | Técnicas vistas |
|---|---|
| **Valores faltantes** | Eliminación (dropna) o imputación (media/mediana/moda) |
| **Outliers** | Detección: Z-score, boxplot · Tratamiento: eliminación con rango intercuartílico (IQR) |
| **Distribuciones sesgadas** | Escalado logarítmico, winsorización (recortar valores extremos) |

Dataset de trabajo: solicitudes de crédito bancario, generado sintéticamente con outliers genuinos inyectados a propósito.

# Valores faltantes: eliminar o imputar según cuánto falta y por qué

Primer paso: medir el porcentaje de nulos por columna con `df.isnull().mean()` y revisar si se concentran en algún grupo.

OPCIÓN 1

## Eliminación

`df.dropna()`

- Borra las filas (o columnas) que tienen valores nulos
- Conviene cuando **faltan pocos datos** y su ausencia es aleatoria
- **Riesgo:** si los nulos se concentran en un grupo, la muestra queda sesgada

OPCIÓN 2

## Imputación

`SimpleImputer(strategy="median")`

- **Media:** variables numéricas con distribución simétrica
- **Mediana:** variables numéricas sesgadas o con outliers
- **Moda:** variables categóricas (el valor más frecuente)
- **Riesgo:** reduce la variabilidad real; con muchos nulos conviene añadir una columna indicadora

**Regla clave:** el valor de imputación se calcula solo con los datos de entrenamiento y se reutiliza tal cual en validación y prueba.

# Outliers: Z-score e IQR detectan los valores extremos con reglas distintas

MÉTODO 1

## Z-score

$$z = \frac{x - \mu}{\sigma}$$

**Outlier si** $|z| > 3$

- Mide a cuántas desviaciones estándar está $x$ de la media
- Supone una distribución aproximadamente normal
- **Limitación:** la media y la desviación se distorsionan con los propios outliers

MÉTODO 2

## Rango intercuartílico (IQR)

$$\mathrm{IQR} = Q_3 - Q_1$$

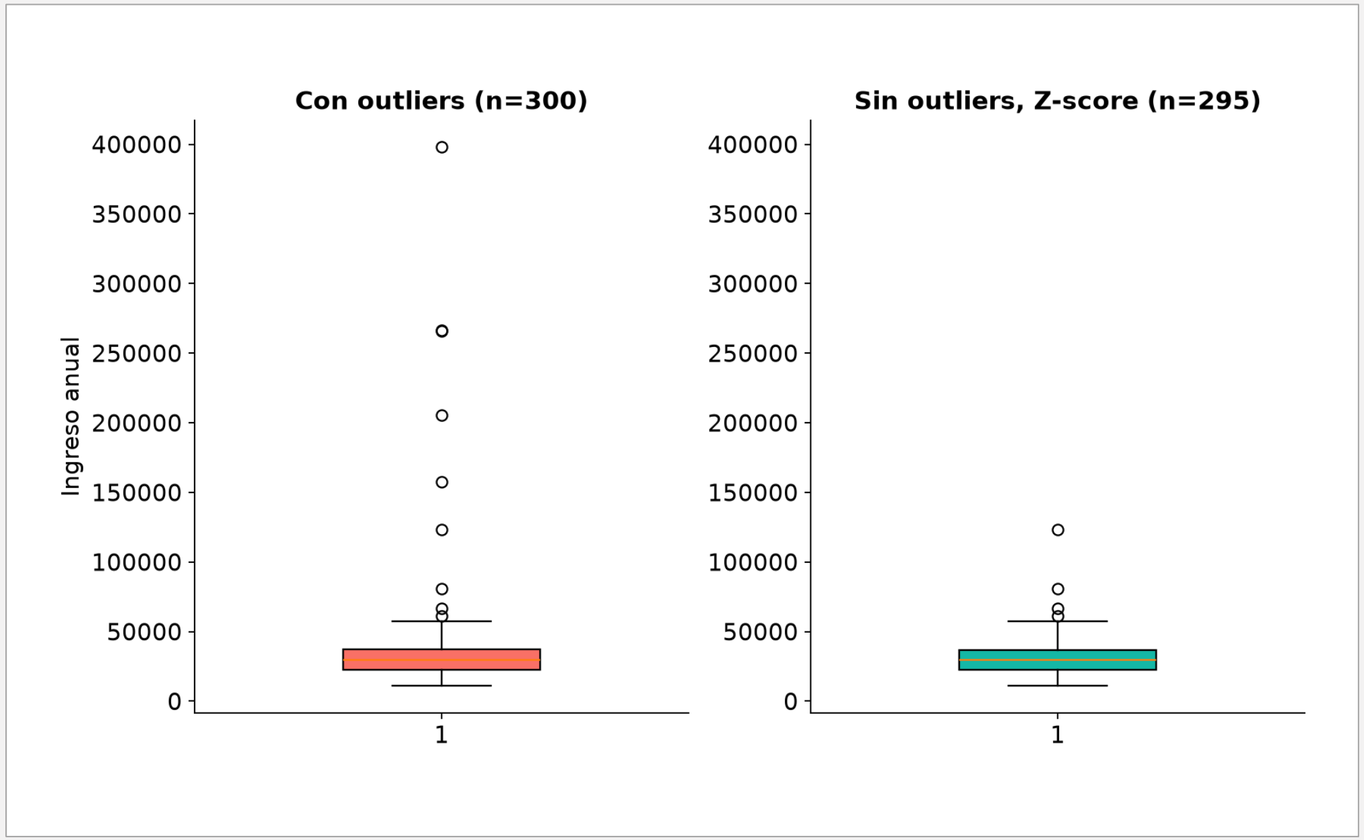
**Outlier si** $x < Q_1 - 1.5 \cdot \mathrm{IQR}$ **o** $x > Q_3 + 1.5 \cdot \mathrm{IQR}$

- Q1 y Q3 son los percentiles 25 y 75 de la variable
- **No depende de la media:** funciona con distribuciones sesgadas
- Es la regla que dibuja los bigotes del boxplot

**Después de detectarlos:** eliminar, recortar (winsorizar) o conservar el valor si es real y relevante para el problema.

# Outliers reales: antes y después del Z-score

- 6 solicitudes con ingresos anormalmente altos (7× el resto) se inyectaron a propósito
- El filtro $|Z| > 3$ los elimina correctamente



# Distribuciones sesgadas: comprimir la cola con logaritmo o recortarla

TRANSFORMACIÓN

## Escalado logarítmico

$$x' = \log(1 + x)$$

- **Comprime los valores grandes** y acerca la distribución a una forma simétrica
- Útil en ingresos, precios y conteos con cola larga a la derecha
- Requiere $x \geq 0$; sumar 1 permite incluir ceros (`np.log1p`)

RECORTE

## Winsorización

$$x' = \min\left(\max\left(x, P_1\right), P_{99}\right)$$

- Reemplaza los valores bajo el percentil 1 y sobre el percentil 99 **por esos mismos límites**
- Conserva todas las filas, a diferencia de eliminar los outliers
- Los percentiles se calculan solo con los datos de entrenamiento

**Por qué importa:** en modelos lineales y basados en distancia, unos pocos valores extremos dominan el ajuste.

# Escalado, codificación, PCA y validación

| Técnica | Qué resuelve |
|---|---|
| **Normalización (Min-Max)** | Lleva cada variable al rango $[0,1]$ |
| **Estandarización (Z-score)** | Media 0, desviación estándar 1 — necesario cuando las variables tienen escalas muy distintas |
| **Codificación ordinal** | Variables categóricas con un orden natural (p. ej. nivel educativo) |
| **One-Hot Encoding** | Variables categóricas sin orden (p. ej. ciudad) |
| **PCA** | Reduce dimensionalidad conservando la mayor varianza posible |
| **Cross-validation / train-val-test** | Evita la fuga de datos (data leakage) entre entrenamiento y evaluación |

Regla de oro: todo transformador (escalador, codificador, PCA) se ajusta (fit) solo con los datos de entrenamiento.

# Min-Max lleva cada variable a [0, 1]; Z-score la centra en 0 con desviación 1

| | Normalización (Min-Max) | Estandarización (Z-score) |
|---|---|---|
| **Fórmula** | $x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$ | $z = \frac{x - \mu}{\sigma}$ |
| **Resultado** | Valores entre 0 y 1 | Media 0 y desviación estándar 1, sin límites fijos |
| **Ante outliers** | Un solo valor extremo comprime al resto en un rango estrecho | Menos afectado: el extremo queda visible como un \|z\| grande |
| **Cuándo usarlo** | Redes neuronales, imágenes y variables con límites conocidos | k-NN, SVM, regresión regularizada y PCA |
| **En scikit-learn** | `MinMaxScaler()` | `StandardScaler()` |

**Sin escalado:** los árboles de decisión y sus ensambles no lo necesitan, porque dividen por umbrales y no por distancias.

# Codificación: ordinal si las categorías tienen orden, One-Hot si no lo tienen

CODIFICACIÓN ORDINAL

## Variables con un orden natural

| nivel_educativo | codificado |
|---|---|
| Primaria | 0 |
| Secundaria | 1 |
| Universitaria | 2 |

```
OrdinalEncoder(categories=[orden])
```

**Los números respetan el orden real:** el modelo puede usar que Universitaria > Primaria

ONE-HOT ENCODING

## Variables sin orden

| ciudad | ciudad_Tunja | ciudad_Duitama | ciudad_Sogamoso |
|---|---|---|---|
| Tunja | **1** | 0 | 0 |
| Duitama | 0 | **1** | 0 |
| Sogamoso | 0 | 0 | **1** |

```
OneHotEncoder(handle_unknown="ignore")
```

**Una columna binaria por categoría:** con cientos de categorías, la dimensión crece y conviene agruparlas

**Error frecuente:** codificar ciudades como 0, 1, 2 hace que el modelo interprete un orden que no existe (Sogamoso > Tunja).

# PCA reduce el número de variables conservando la mayor varianza posible

**01**

### Estandarizar

Cada variable a media 0 y desviación 1, para que ninguna domine por su escala

▶

**02**

### Calcular la covarianza

Mide cómo varían juntas las variables entre sí

▶

**03**

### Obtener componentes

Direcciones ortogonales de máxima varianza (vectores propios)

▶

**04**

### Elegir k componentes

Las primeras que sumen, por ejemplo, el 95% de la varianza explicada

## Qué se obtiene

- Cada componente es una **combinación lineal** de las variables originales
- PC1 conserva más varianza que PC2, y así sucesivamente

## Cuándo usarlo y qué cuesta

- Visualizar datos en 2D, acelerar modelos y reducir la colinealidad
- **Costo:** los componentes pierden el significado directo de las variables

```
PCA(n_components=0.95)
```

# Separar los datos antes de ajustar cualquier transformador evita la fuga de datos

## PARTICIÓN ENTRENAMIENTO / VALIDACIÓN / PRUEBA

**Entrenamiento · 70%**
`fit + transform`

Se ajustan el modelo y todos los transformadores: imputador, escalador, codificador y PCA

**Validación · 15%**
`solo transform`

Elegir hiperparámetros

**Prueba · 15%**
`solo transform`

Evaluación final, una sola vez

## VALIDACIÓN CRUZADA (k = 5)



En cada ronda, una partición (rojo) valida y las otras cuatro entrenan.

La métrica final es el **promedio de las 5 rondas**

## FUGA DE DATOS

Si el escalador se ajusta con todo el dataset, la media y la desviación de prueba se filtran al entrenamiento y **la métrica resulta optimista.**

**Solución:** `Pipeline` + `cross_val_score` ajusta cada paso dentro de cada ronda.

06

# Introducción a Machine Learning

# Aprendizaje supervisado: regresión y clasificación

| Tarea | Modelos vistos | Dataset de ejemplo |
|---|---|---|
| **Regresión** | Regresión lineal | California Housing (predecir precio de vivienda) |
| **Clasificación** | k-NN, Árboles de Decisión | Iris (4 variables, 3 clases bien separadas) |

k-NN clasifica un punto nuevo según la clase mayoritaria entre sus k vecinos más cercanos.

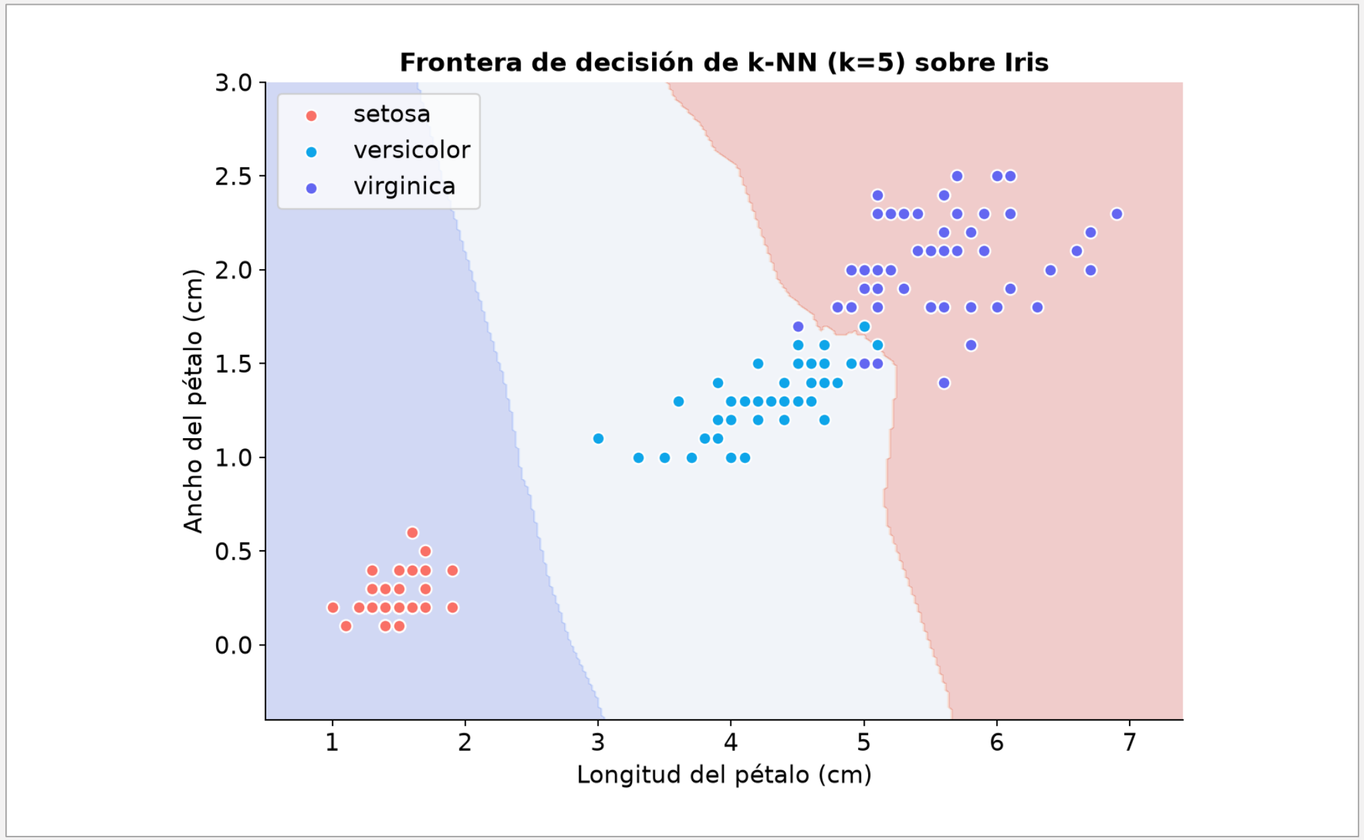
# Significado de las métricas de evaluación en regresión

- **MAE (Error Absoluto Medio):** Mide el promedio de los errores en las mismas unidades que la variable original, sin importar la dirección del error. Es poco sensible a valores atípicos.
- **MSE (Error Cuadrático Medio):** Calcula el promedio de los errores al cuadrado, penalizando fuertemente los errores grandes. Sus unidades quedan elevadas al cuadrado.
- **RMSE (Raíz del Error Cuadrático Medio):** Es la raíz cuadrada del MSE. Devuelve el error a las unidades originales, manteniendo la penalización a los errores grandes.
- $R^2$ **(Coeficiente de Determinación):** Indica qué porcentaje de la variabilidad de los datos reales logra ser explicada por el modelo. Varía entre 0 y 1 (donde 1 es un ajuste perfecto).

# La frontera de decisión de k-NN, en la práctica

$k = 5$: cada región del plano se colorea según la clase mayoritaria entre los 5 vecinos más cercanos

Con Iris, incluso una regla tan simple separa casi perfectamente las 3 especies



# Significado de las métricas de evaluación en clasificación

- **Accuracy (Exactitud):** Es una métrica global que evalúa el total de predicciones correctas sobre el conjunto completo de datos.
- **Precision (Precisión):** De las veces que el modelo predijo esa clase, cuántas acertó. Se calcula de manera individual para cada clase y responde a preguntas más específicas sobre el rendimiento del modelo.
- **Recall (Exhaustividad):** De todos los ejemplos que realmente eran de esa clase, a cuántos identificó.
- **F1-score:** Es la media armónica de la precisión y recall. Combina estas dos medidas en un único valor que evalúa el rendimiento especialmente cuando se trabaja con conjuntos de datos desequilibrados. Esto asegura que un modelo solo obtenga una puntuación alta si tanto su precisión como su exhaustividad son razonablemente elevadas.
- **Desbalance de clases:** Cuando las clases están desbalanceadas, dos modelos con la misma exactitud pueden tener resultados muy diferentes de precisión y recall por clase.

# Cada métrica se calcula con celdas distintas de la matriz de confusión




**Accuracy** $\frac{VP+VN}{Total}$

Proporción de aciertos sobre todas las predicciones

**Precision** $\frac{VP}{VP+FP}$

De lo que el modelo predijo positivo, cuánto lo era

**Recall** $\frac{VP}{VP+FN}$

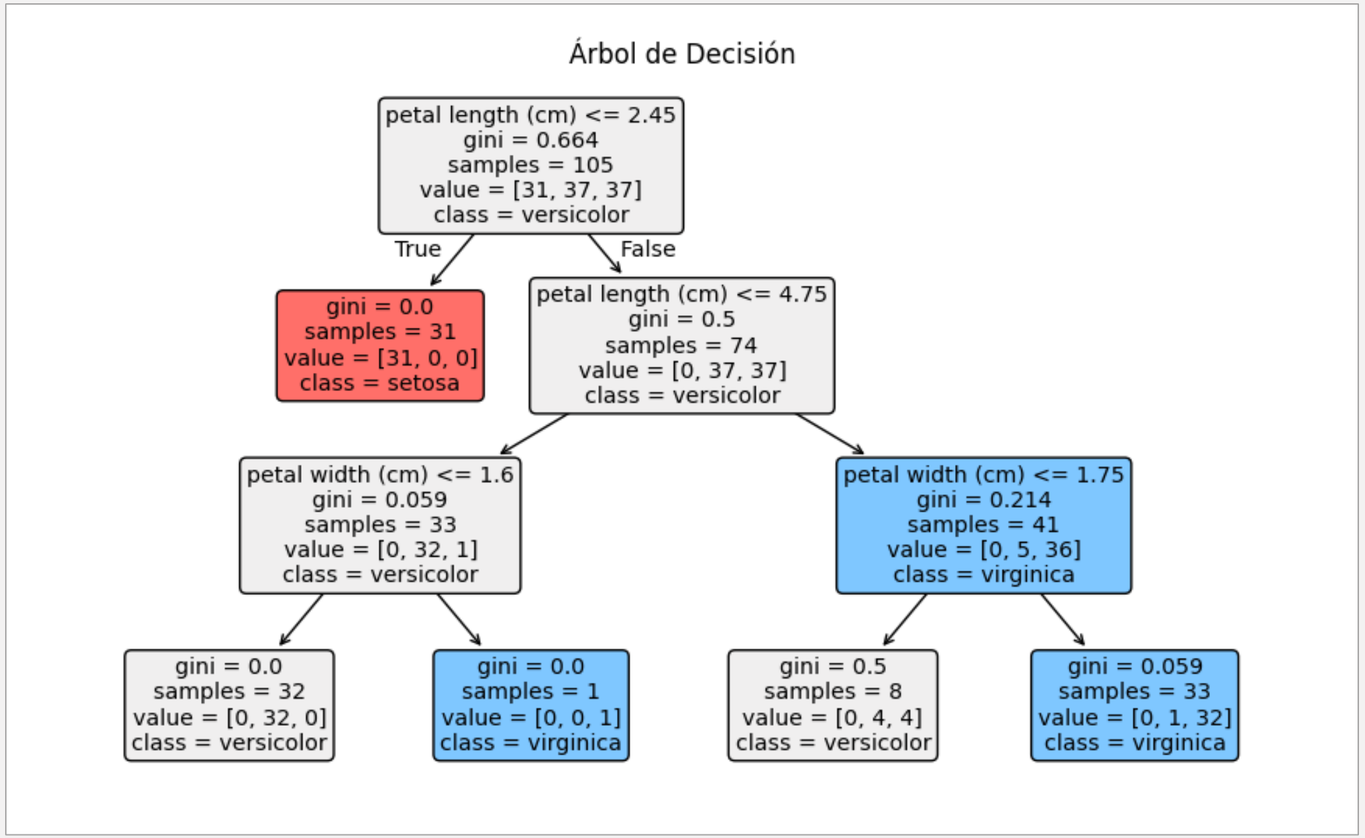
De lo que realmente era positivo, cuánto detectó

**F1-score** $\frac{2 \cdot P \cdot R}{P+R}$

Media armónica de precision y recall: usa VP, FP y FN

# La interpretabilidad de los árboles de decisión

- **Alta Interpretabilidad:** Permiten seguir el camino de decisión desde la raíz hasta las hojas para entender cada clasificación.
- **Criterio de Gini:** Miden y reducen la impureza en cada división hasta alcanzar nodos homogéneos.
- **Control de Complejidad:** Limitar la profundidad máxima previene el sobreajuste y mantiene el modelo simple.

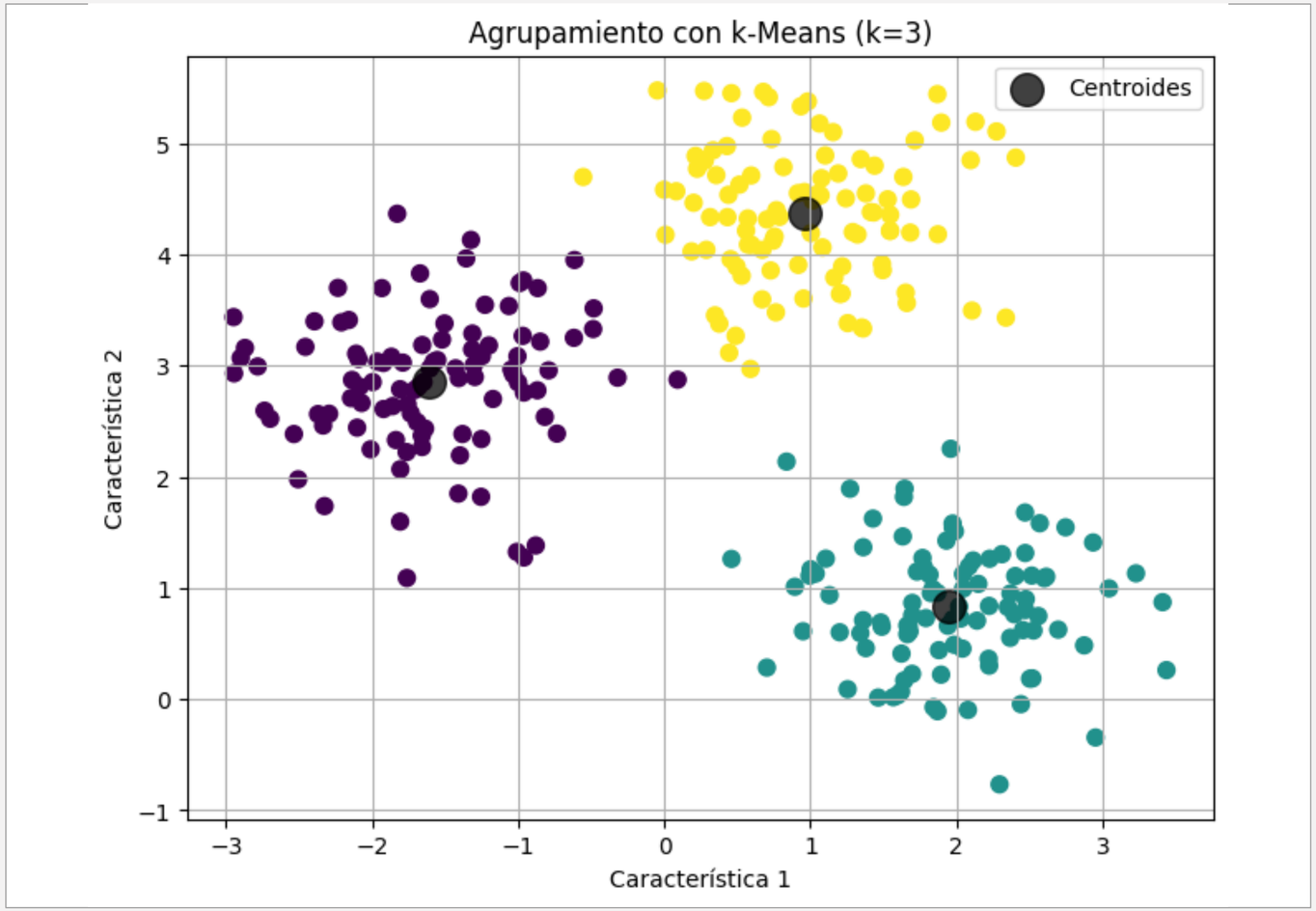


# Aprendizaje No Supervizado: Clustering con K-Means

El coeficiente de Silhouette evalúa la calidad de un agrupamiento (clustering) sin etiquetas, midiendo la cohesión interna frente a la separación con otros clústeres.

Interpretación de sus valores (rango de -1 a +1):

- Cercano a +1: Agrupamiento excelente.
- Cercano a 0: Clústeres superpuestos.
- Cercano a -1: Asignación incorrecta.



# No supervisado, ensambles y AutoML

| Bloque | Qué se ve |
|---|---|
| **No supervisado** | Clustering con K-Means — agrupar datos sin etiquetas previas |
| **Ensambles** | Random Forest (bagging de árboles), XGBoost (boosting) |
| **AutoML** | PyCaret: compara automáticamente docenas de modelos con pocas líneas de código |
| **Optimización de hiperparámetros** | Grid Search (prueba exhaustiva) y Randomized Search (muestreo aleatorio del espacio de búsqueda) |

Este notebook es el recorrido panorámico del curso: cada bloque se retoma y profundiza en los notebooks siguientes.

07

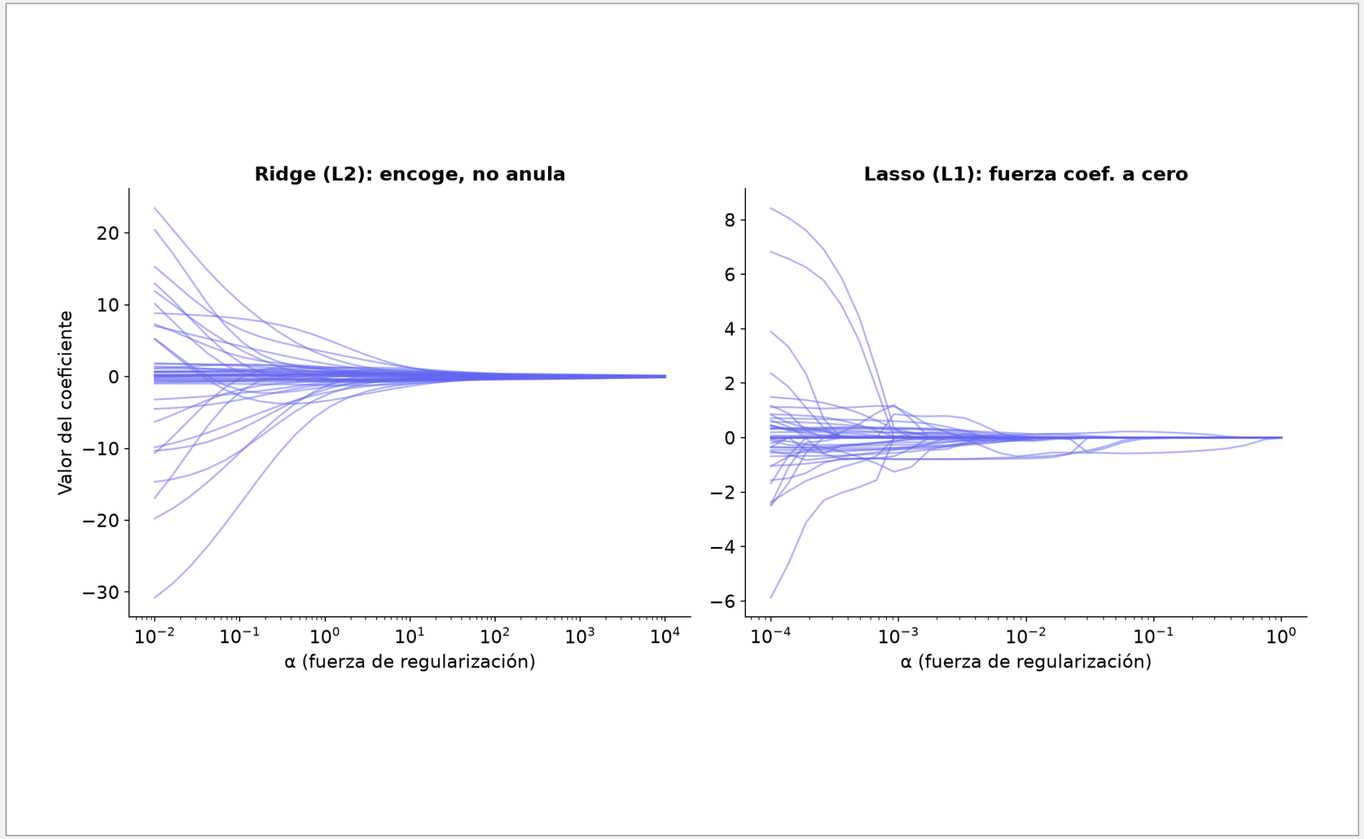
# Aprendizaje Supervisado: Regresión Avanzada

# Sobreajuste y regularización

- Un ejemplo sintético 1D muestra el sobreajuste de forma aislada: un polinomio de grado alto se ajusta perfecto a los puntos de entrenamiento, pero oscila entre ellos
- Sobre California Housing (expandido con PolynomialFeatures de grado 2) se comparan tres formas de regularizar:
- Ridge (L2): penaliza el cuadrado de los coeficientes, los encoge sin anularlos
- Lasso (L1): penaliza el valor absoluto — puede llevar coeficientes a exactamente cero
- ElasticNet: combina L1 y L2 en una sola penalización

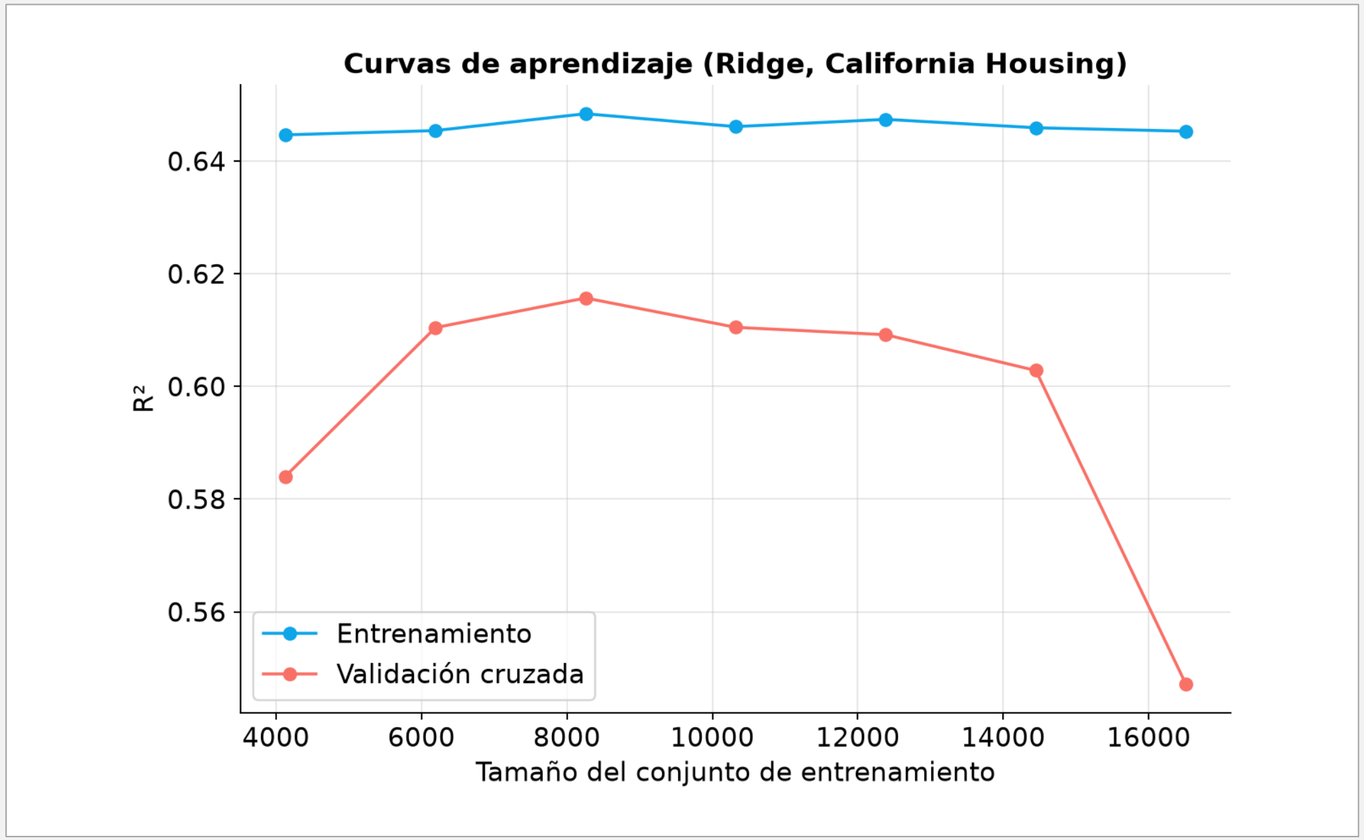
# El efecto real de Ridge vs. Lasso

- Mismos datos (California Housing + interacciones polinomiales) — cada panel usa el rango de α donde su propio efecto se aprecia mejor
- Lasso lleva varios coeficientes a exactamente cero a medida que α crece; Ridge los encoge, pero rara vez llega a cero



# Curvas de aprendizaje reales (Ridge)

- Brecha entre entrenamiento y validación cruzada: diagnóstico visual de sesgo/varianza
- Calculado sobre datos reales de California Housing



08

# Aprendizaje Supervisado: Clasificación Avanzada

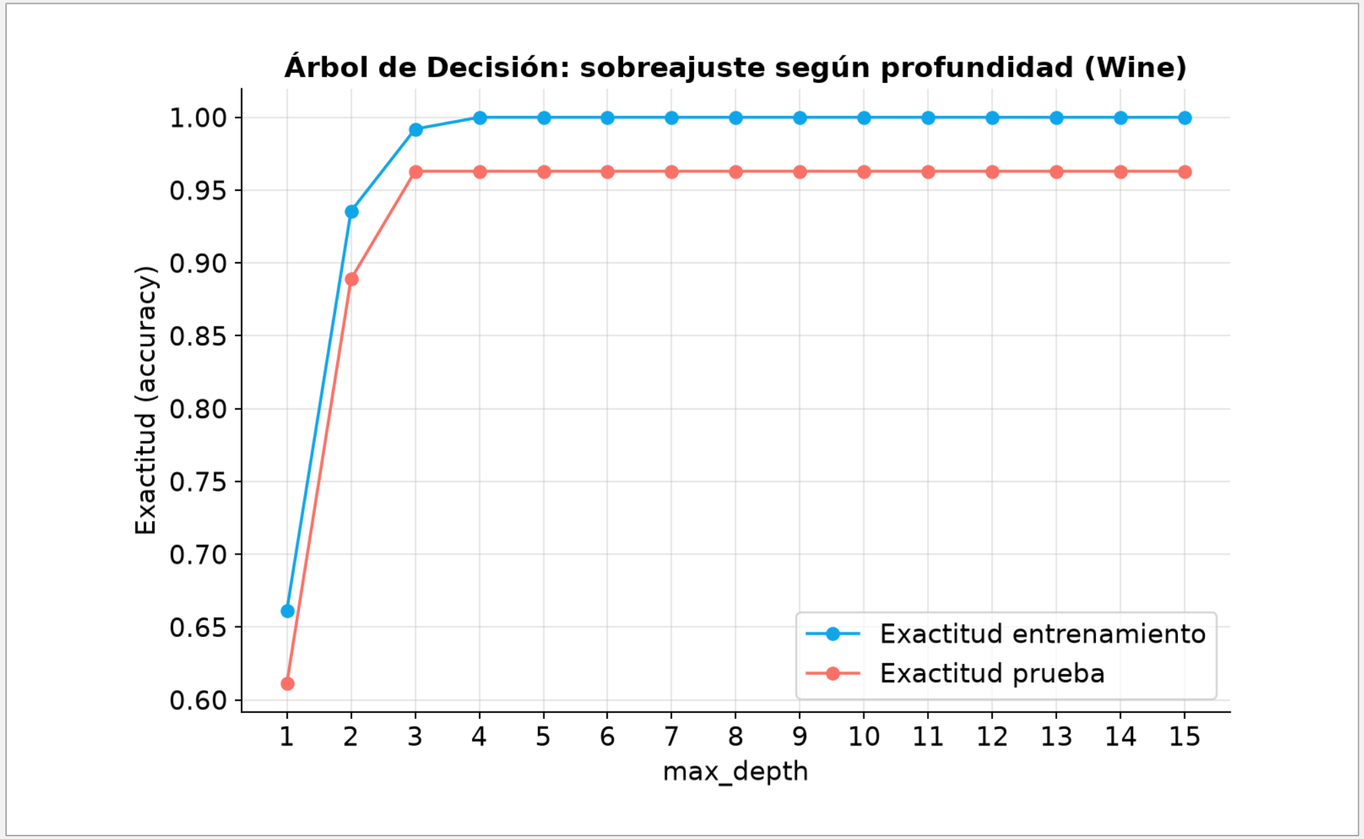
# Por qué Wine y no Iris, esta vez

Iris tiene solo 4 variables y sus 3 clases están casi perfectamente separadas — varios modelos llegan al 100% de exactitud sin esfuerzo. Este notebook usa en su lugar Wine (13 variables), donde sí se aprecian diferencias entre modelos.

| Modelo | Idea central |
|---|---|
| **Regresión logística (multiclase)** | Extiende la regresión logística binaria a más de dos clases |
| **SVM (kernel RBF)** | Busca el hiperplano (o frontera no lineal) que separa las clases con el mayor margen posible |
| **Árbol de Decisión** | Se muestra su curva de sobreajuste: a mayor profundidad, más se ajusta al ruido |

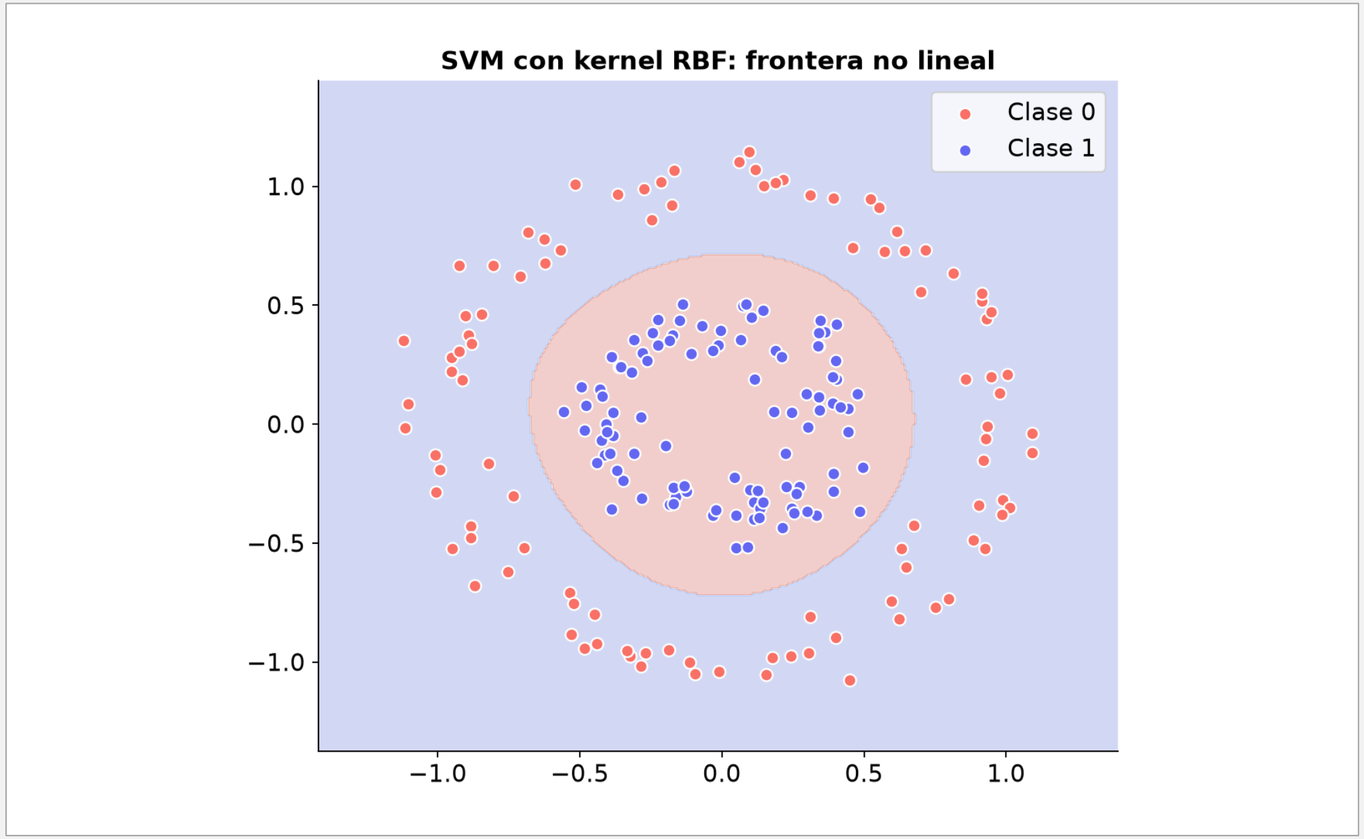
# Árbol de Decisión: sobreajuste real, sobre Wine

- max_depth ≈ 3-4: mejor exactitud de prueba
- Profundidades mayores solo memorizan el entrenamiento



# SVM: fronteras no lineales con el kernel RBF

- Ejemplo ilustrativo en 2D: un kernel lineal no podría separar estas dos clases
- El kernel RBF proyecta implícitamente a más dimensiones, donde sí existe un separador simple



# Comparación final con validación cruzada

- En el notebook principal, k-NN y los Árboles se evaluaron con una única partición de entrenamiento y prueba
- Aquí se comparan cuatro modelos con cross_val_score (5 particiones), sobre las 13 variables de Wine: Regresión Logística, SVM (kernel RBF), Árbol de Decisión y k-NN ($k = 5$)
- Todos con la misma preparación de datos (StandardScaler) — una comparación justa entre algoritmos

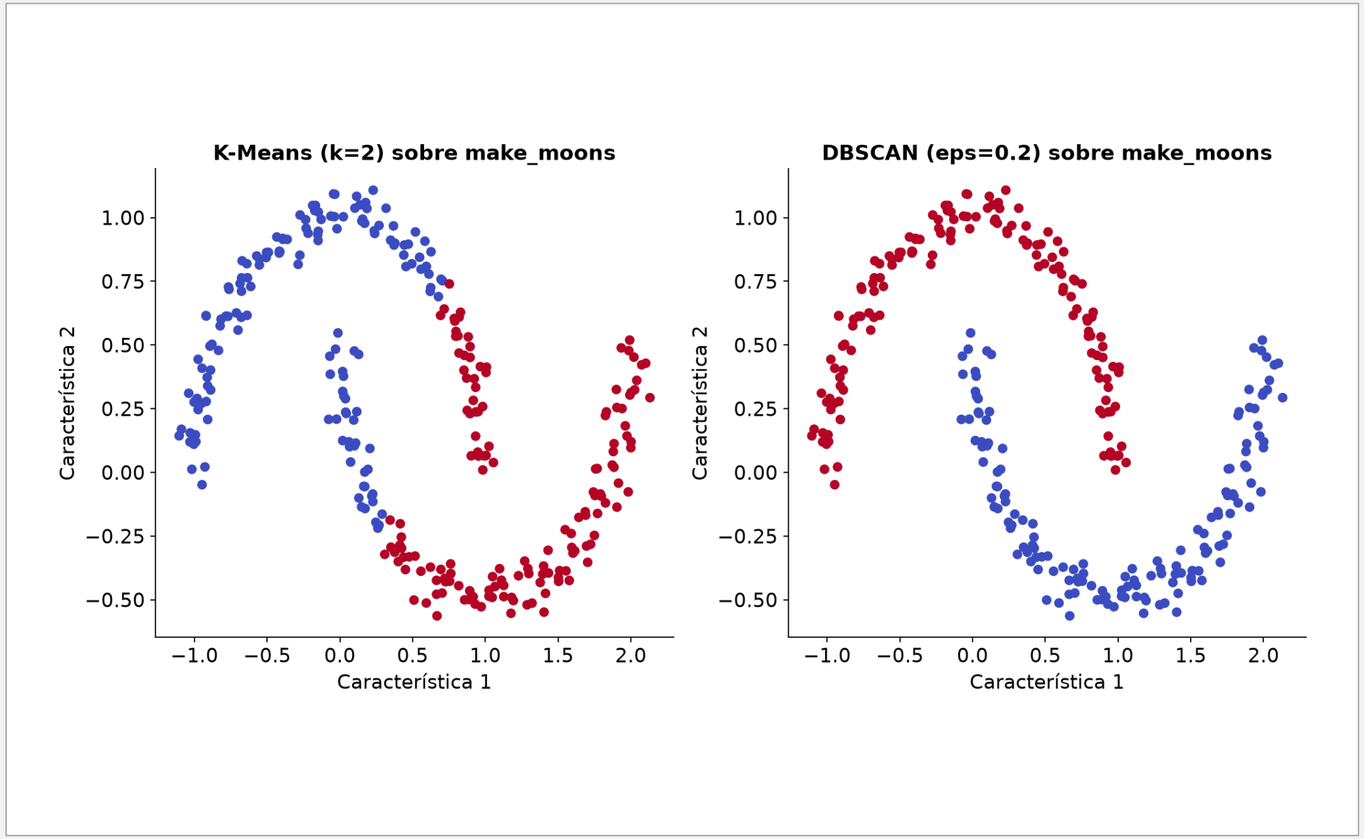
09

# Aprendizaje No Supervisado

09

# K-Means falla, DBSCAN lo resuelve

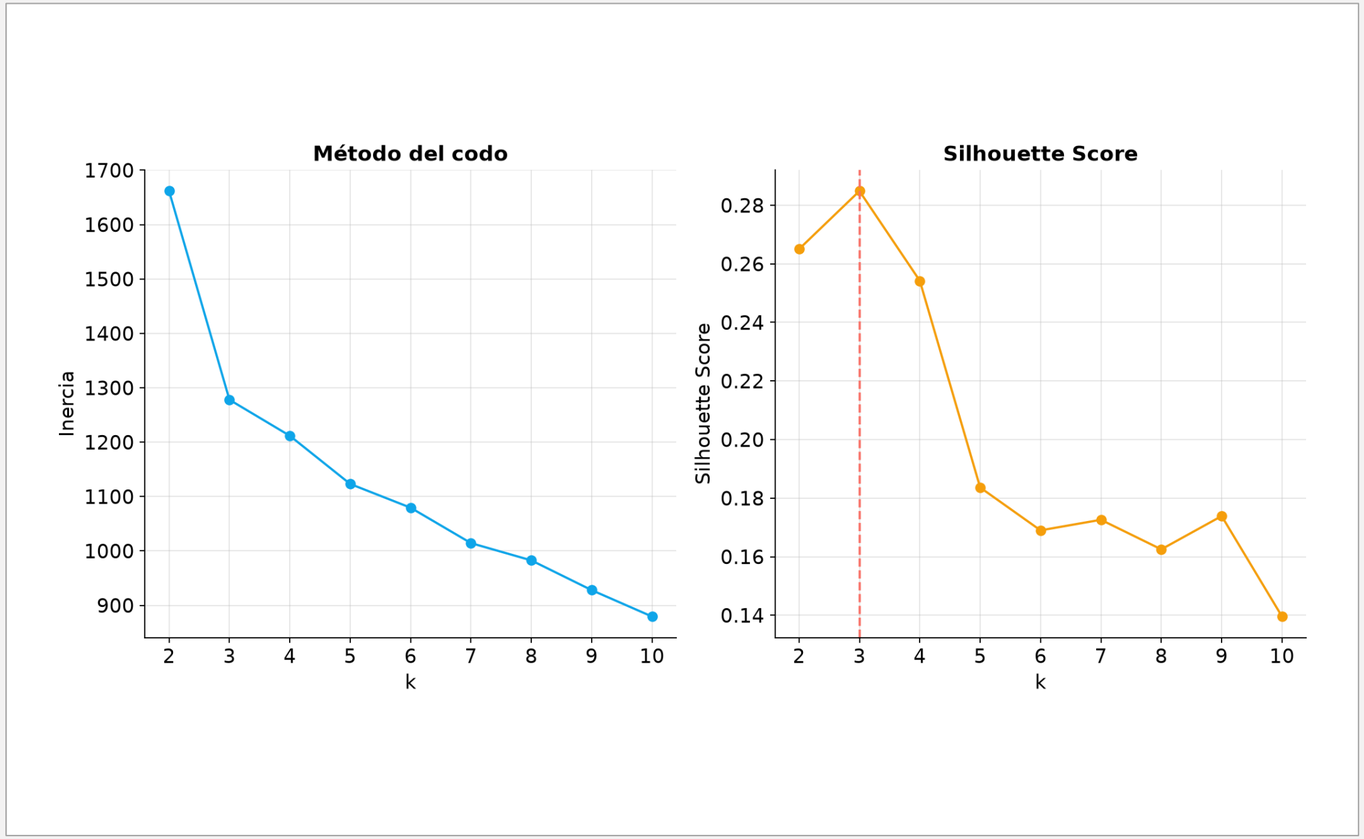
- Mismos datos (make_moons), mismo $k = 2$ — K-Means traza una frontera recta que corta cada media luna por la mitad
- DBSCAN agrupa por densidad, no por cercanía a un centroide, y sí recupera la forma real de los dos grupos



# Elegir k de forma objetiva, sobre Wine real

El mayor Silhouette Score real ocurre en $k = 3$ (score=0.285) — coincide exactamente con las 3 variedades de uva reales del dataset Wine

El codo (inercia) es más ambiguo aquí; el Silhouette Score da una respuesta más objetiva



10

# Modelos de Ensamble

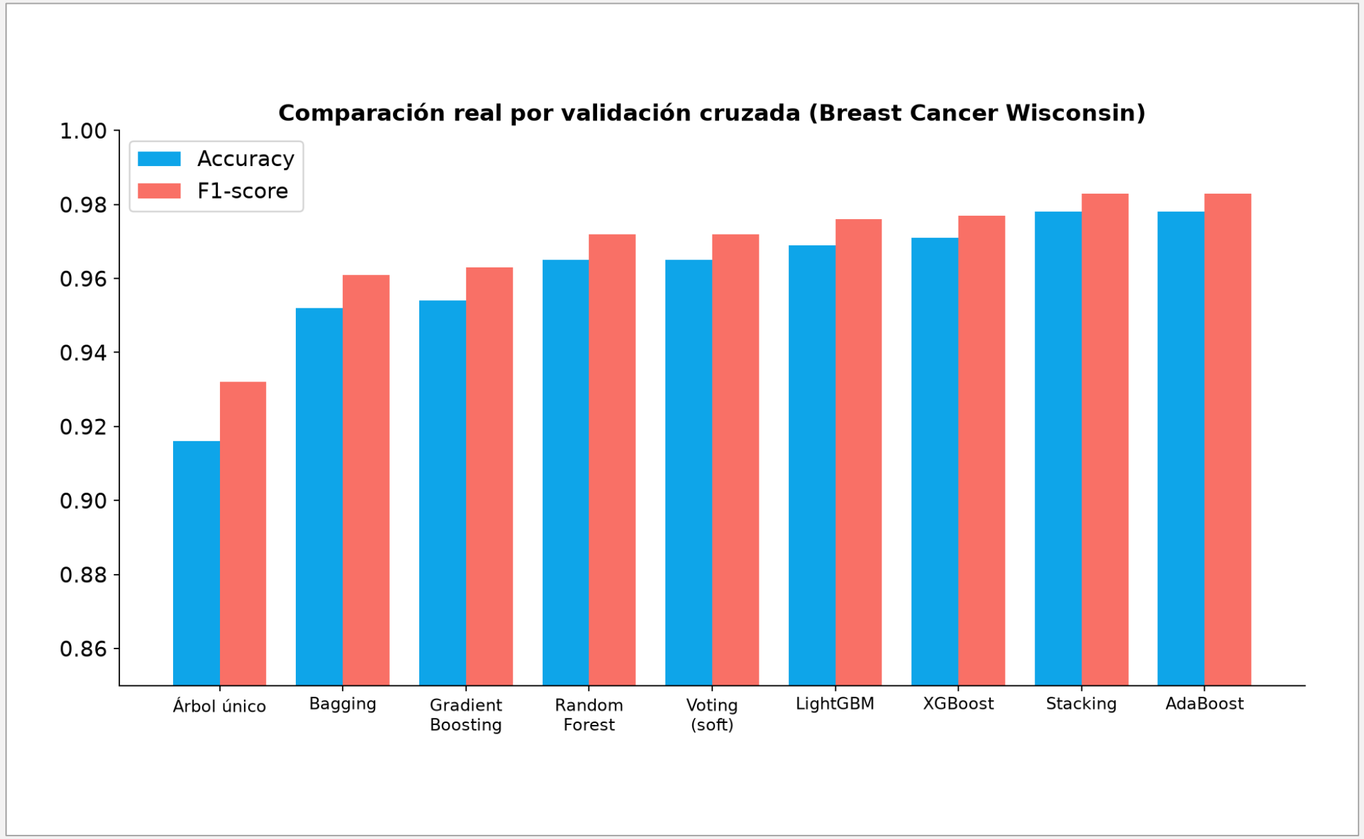
# Bagging, Boosting, Voting y Stacking

| Familia | Modelos vistos |
|---|---|
| **Bagging** | Bagging genérico, Random Forest (con importancia de características) |
| **Boosting** | AdaBoost, Gradient Boosting, XGBoost, LightGBM |
| **Voting** | Combina las predicciones de varios modelos ya entrenados (votación dura o suave) |
| **Stacking** | Un meta-modelo aprende a combinar las salidas de los modelos base |

Todo se evalúa sobre el mismo dataset — Breast Cancer Wisconsin (diagnóstico de tumores).

# Comparación real, por validación cruzada

- Hasta el ensamble más simple (Bagging) supera claramente al árbol único
- AdaBoost y Stacking lideran con ROC-AUC ≈ 0.995



11

# Pipelines y Evaluación de Modelos

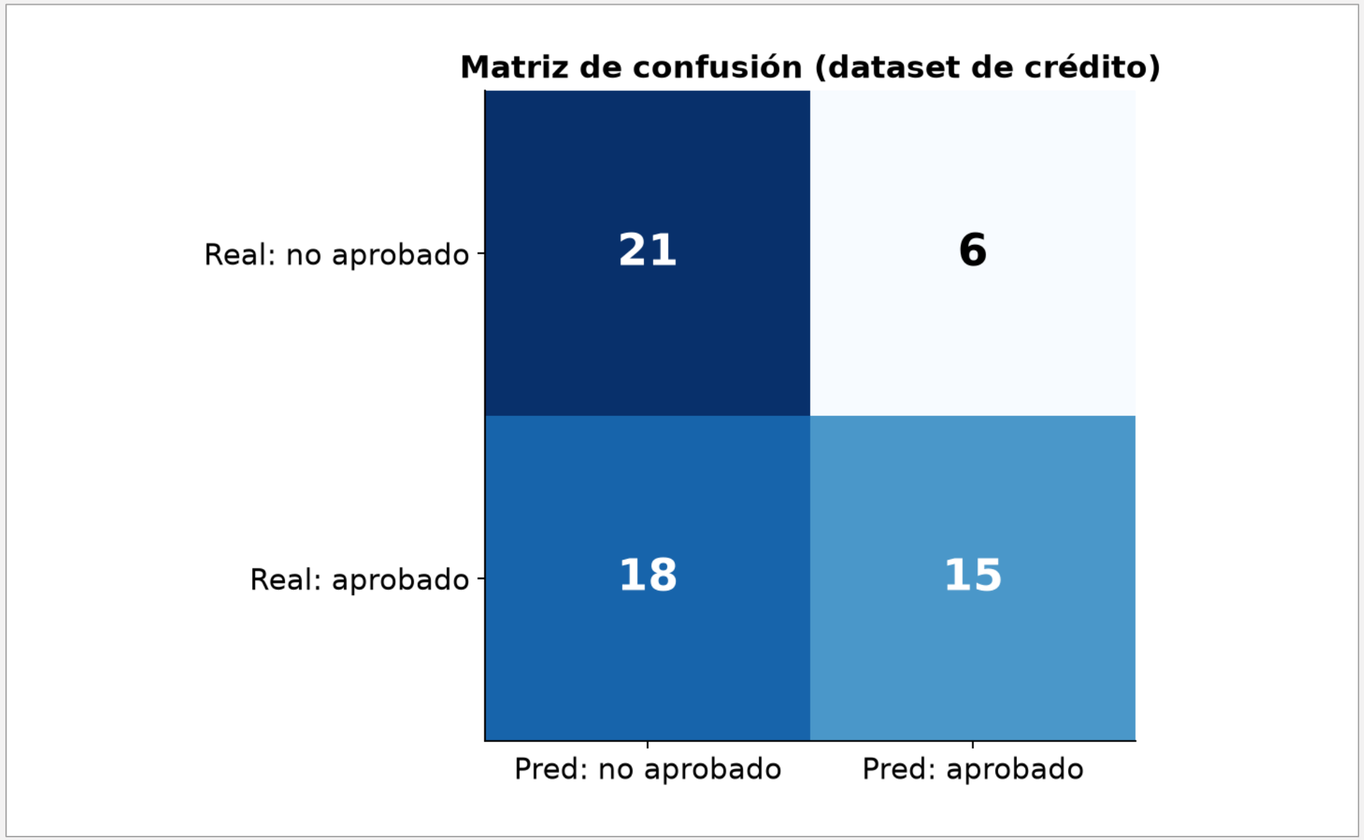
# Pipeline y ColumnTransformer

- Un Pipeline encadena pasos (imputar → escalar → codificar → clasificar) como un único estimador con fit/predict/score
- Evita la fuga de datos por construcción: cada transformación se ajusta solo con los datos de entrenamiento
- ColumnTransformer aplica transformaciones distintas según el tipo de columna, en el dataset de crédito bancario:

  numéricas (edad, ingreso, puntaje) → imputar + escalar · categóricas (ciudad, nivel educativo) → codificar
- Sobre ese Pipeline completo se aplican validación cruzada y GridSearchCV

# Métricas reales, más allá de accuracy

- Clase "aprobado": precisión 0.71, recall 0.45 · Clase "no aprobado": precisión 0.54, recall 0.78
- Accuracy global: 0.60 — ROC-AUC: 0.692
- La accuracy global no deja ver que el modelo detecta solo el 45% de las solicitudes que debían aprobarse



# Clases desbalanceadas y SMOTE

- Con un dataset deliberadamente desbalanceado (95% / 5%), un clasificador "ingenuo" alcanza ~95% de accuracy sin haber aprendido nada
- SMOTE genera ejemplos sintéticos de la clase minoritaria, interpolando entre vecinos reales, en vez de solo duplicarlos
- Curvas de aprendizaje y de validación: mismo diagnóstico de sesgo/varianza de la Unidad 3.7, ahora dentro de un Pipeline completo

# Resumen de la Unidad 3

| Tema | Notebook del curso |
|---|---|
| **Álgebra lineal, gradiente, Bayes** | 1-AI-fundamentals/1, 2, 3 |
| **EDA y preprocesamiento** | 1-AI-fundamentals/4, 5 |
| **Recorrido completo de ML** | 2-introduccion-machine-learning.ipynb |
| **Regresión y clasificación avanzada** | 3-aprendizaje-supervisado/1, 2 |
| **Aprendizaje no supervisado** | 4-aprendizaje-no-supervisado.ipynb |
| **Modelos de ensamble** | 5-modelos-ensamble.ipynb |
| **Pipelines y evaluación** | 6-pipelines-evaluacion.ipynb |

# ¡Gracias!

Estos algoritmos y buenas prácticas son la base del Machine Learning clásico — y la comparación directa frente a la Unidad 4 (redes neuronales).